

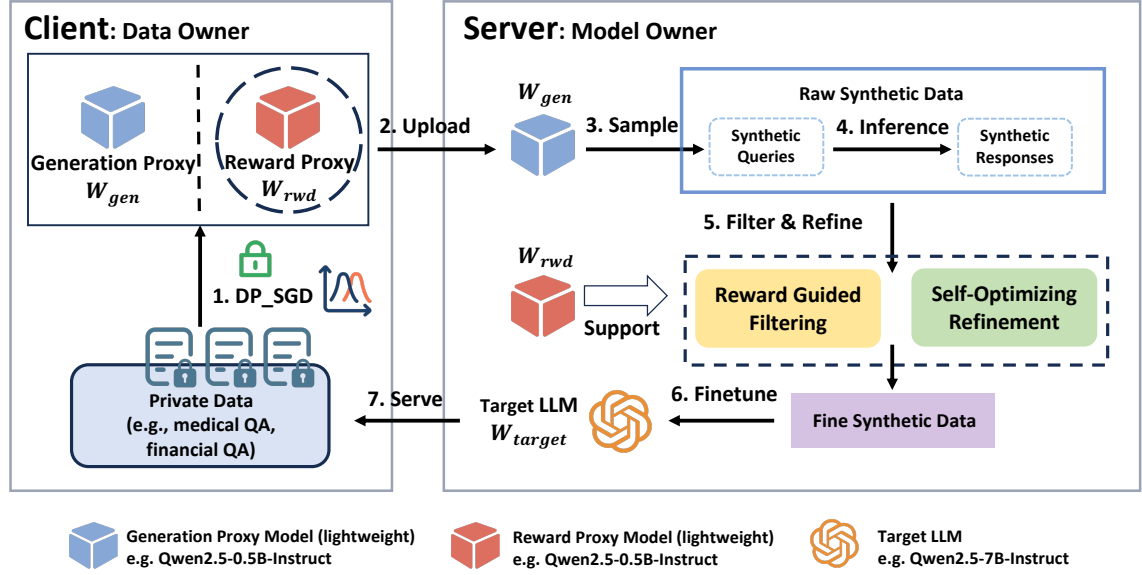


Figure 2: The overview of our *RewardDS* framework. The client uses DP-SGD to fine-tune two lightweight proxy models on privacy-sensitive data: the Generation Proxy Model W_{gen} and the Reward Proxy Model W_{rwd} . Both proxy models are then sent to the server. The Generation Proxy Model is used to sample raw synthetic data, consisting of queries and responses. The Reward Proxy Model supports the **Reward Guided Filtering** and **Self-Optimizing Refinement** modules, which filter and refine the raw synthetic data to produce fine synthetic data. Finally, the target LLM W_{target} is fine-tuned on the fine synthetic data and provides service to the client for domain-specific tasks.

4.2 Server Side

Synthetic Data Generation. Following Yu et al. (2024); Wang et al. (2024), we use W_{gen} to generate both synthetic queries and their corresponding responses, collectively referred to as raw synthetic data. Although the generation proxy model W_{gen} is trained on private data and learns domain-specific knowledge, the generation process of raw synthetic data is random and unstable. As a result, the raw synthetic data inevitably contains noisy samples, and fine-tuning the LLM directly on this data can lead to performance degradation.

Reward Guided Filtering. We leverage the reward proxy model W_{rwd} to evaluate each synthetic data and filter out those with low rewards. A lower reward indicates a higher likelihood of the synthetic data being noisy. We select only the top $\lfloor L/k \rfloor$ data, where L is the total number of synthetic data and k is the partition fold (Line 10 in Alg. 1). To compensate for the reduced synthetic dataset size after filtering, we replicate the high-reward data to maintain the total data volume during the target LLM fine-tuning (Line 11 in Alg. 1).

Self-Optimizing Refinement. While filtering mitigates noise, it selects only a small subset of samples, potentially leading to overfitting on limited data. Building on LLMs’ self-reflection ca-

pabilities (Madaan et al., 2023), we implement a dynamic data refinement strategy to improve low-reward samples, enhancing overall data quality. Initially, for each synthetic query, we generate N candidate responses rather than only one response using the generation proxy model (Line 3 in Alg. 1). The reward proxy model then selects the response with the highest reward score as the chosen response (Line 5 in Alg. 1). We directly fine-tune the target LLM W_{target} on the chosen response (Line 16 in Alg. 1).

After fine-tuning the target LLM W_{target} for each epoch, we dynamically refine the synthetic data for the next epoch’s training. For each query’s N candidate responses, we identify the lowest-reward responses and combine them with the highest-reward responses to form the rejected (A_r) and chosen (A_c) response pairs. The generation proxy model M_{gen} analyzes these responses and provides feedback, highlighting the strengths of A_c and weaknesses of A_r (Line 18). This feedback, along with the original query, guides the target LLM W_{target} to generate N refined candidate responses (Line 19). Finally, the reward proxy model selects the highest-reward response from these refined candidates for the next epoch’s LLM fine-tuning (Line 20).

The collaborative process between the reward-guided filtering and self-optimizing refinement

Algorithm 1 *RewardDS* based LLM Fine-tuning

Input: Synthetic query set $\mathcal{Q}_{\text{query}}$, number of synthetic query L , number of candidate responses N , partition fold k , generation proxy model W_{gen} , reward proxy model W_{rd} , target LLM W_{target} , training epoch T

Output: The fine-tuned LLM W_{target}^T

```

1 // Before Fine-tuning LLM
2 for each query  $q \in \mathcal{Q}_{\text{query}}$  do
3   Generate candidate response set:  $\{A_j\}_{j=1}^N \leftarrow W_{\text{gen}}(q)$ 
4   Predict the reward score:  $\{s_j\}_{j=1}^N \leftarrow W_{\text{rd}}(q, A_j)$ 
5   Select the best and the worst response:
6      $(A_c, A_r) \leftarrow (A_{\arg \max_j s_j}, A_{\arg \min_j s_j})$ 
7   Record the best reward score:  $s_c \leftarrow \max_j s_j$ 
8   Gather the initial synthetic dataset:  $\mathcal{D}_0 \leftarrow \{(q_i, A_c^i, A_r^i, s_c^i)\}_{i=1}^L$ 
9   Sort  $\mathcal{D}_0$  by reward:  $\mathcal{D}_0^{\text{sorted}} \leftarrow \{(q_i, A_c^i, A_r^i, s_c^i)\}_{i=1}^L$ , where
      $s_c^1 \geq \dots \geq s_c^L$ 
10  Partition  $\mathcal{D}_0^{\text{sorted}}$  into  $k$  folds:  $\{\mathcal{D}_0^m\}_{m=1}^k \leftarrow \text{split}(\mathcal{D}_0^{\text{sorted}}, k)$ 
11  Extract top- $\lfloor L/k \rfloor$  samples:  $\mathcal{D}_{\text{high}} \leftarrow \mathcal{D}_0^1$ 
12  Replicate subset to obtain the train set:  $\mathcal{T}_0 \leftarrow \bigoplus_{\lceil L/|\mathcal{D}_{\text{high}}| \rceil} \mathcal{D}_{\text{high}}$ 
13  Determine score threshold  $\tau$ :  $\tau \leftarrow \min_{s_c \in \mathcal{D}_{\text{high}}} s_c$ 
14 // During Fine-tuning LLM
15 Initialize target LLM:  $W_{\text{target}}^0 \leftarrow W_{\text{target}}$ 
16 for iteration  $t = 1$  to  $T$  do
17   Fine-tune target LLM  $W_{\text{target}}^{t-1}$  on  $\{(q, A_c) \in \mathcal{T}_{t-1}\}$  and get
      $W_{\text{target}}^t$ 
18   for each query  $q \in \mathcal{Q}_{\text{query}}$  do
19     Generate feedback  $\phi$ :  $\phi \leftarrow W_{\text{gen}}(q, A_c, A_r)$ 
20     Re-generate the candidate response set:
        $\{A_j\}_{j=1}^N \leftarrow W_{\text{target}}^t(q, \phi)$ 
21     Predict the reward score, select the best and worst responses, and record the highest reward score to update
        $\mathcal{D}_{t-1}$ , yielding  $\mathcal{D}_t$ .
22   Filter and get new training set  $\mathcal{T}_t$ :
      $\mathcal{T}_t \leftarrow \{(q, A_c, A_r, s_c) \in \mathcal{D}_t \mid s_c \geq \tau\}$ 
23 return  $M_{\text{target}}^T$ 

```

modules is presented in Alg. 1. The refinement instruction templates are provided in Appendix J. After the LLM is fine-tuned on the refined synthetic data, it can provide service to the client for those domain-specific tasks.

5 Privacy Analysis

The only transmitted contents between the client and server are the generation proxy model and the reward proxy model. Both models are fine-tuned on the private dataset using the DP-SGD algorithm (Abadi et al., 2016). According to the definition of differential privacy (DP) (Dwork and Roth, 2014), adversaries cannot infer any private data from the fine-tuned proxy models. Additionally, based on the post-processing property of the DP framework (Dwork and Roth, 2014), any further operations on the two proxy models will not cause privacy leakage. All subsequent operations on the server, including synthetic data generation, reward-guided filtering, and self-optimizing refinement, are privacy-preserving. Moreover, we conduct Data Extraction Attack (Carlini et al., 2021) and Membership Inference Attack (Yeom et al.,

2018; Choquette-Choo et al., 2021) on our method to empirically demonstrate its privacy protection capability in Section 6.4.

We have fine-tuned two proxy models on the private dataset and the privacy budget of each fine-tuning is (ϵ, δ) . According to the sequential composition law of DP mechanism (Dwork and Roth, 2014), the total privacy budget of our framework is $(2\epsilon, 2\delta)$.

6 Experiments

6.1 Experiments Setup

Datasets. We evaluate our method across three domain-specific generation tasks using established datasets: Medical QA using HealthCareMagic-100k (Li et al., 2023), Financial QA using fingpt-fqa_qa (Zhang et al., 2023), and Code Generation using opc-sft-stage2 (Huang et al., 2024).

Evaluation Metrics. For the evaluation of the QA task, we employ the ROUGE-1 (R1), ROUGE-L (RL) (Lin, 2004), and Perplexity (PPL) (Hu et al., 2024) as metrics. While automated metrics focus on lexical overlap and fluency, we also use LLM-Judge (Zheng et al., 2023) to provide a more comprehensive assessment of semantic accuracy and response quality. For the code generation task, we use Pass@1 and Pass@10 as evaluation metrics (Chen et al., 2021).

Implementation Details. We use the Qwen2.5-0.5B-Instruct model (Yang et al., 2024b) as the backbone for the generation/reward proxy model, and the Qwen2.5-7B-Instruct model as the target LLM on the server. During each DP-SGD fine-tuning process of both proxy models, we set the privacy budget to $(8, 1e^{-5})$. As a result, the total privacy budget for our method is $(16, 2e^{-5})$, according to the sequential composition law of the DP mechanism (Abadi et al., 2016). For a fair comparison, we set the same privacy budget for all compared methods. The size of the synthetic dataset is always kept to twice that of the client’s private data across all baselines. These settings align with established DP deployments such as Apple’s QuickType and Google’s models, as noted by Lukas et al. (2023).

More details on the datasets used and the implementation are provided in Appendix B and Appendix D, respectively.

Methods	Medical QA			Financial QA			Code Generation	
	R1 $\uparrow$	RL $\uparrow$	PPL $\downarrow$	R1 $\uparrow$	RL $\uparrow$	PPL $\downarrow$	Pass@1 $\uparrow$	Pass@10 $\uparrow$
Vanilla LLM	21.60	11.50	1.34	23.91	11.72	1.38	18.82	42.06
Locally Fine-tuning	<u>23.82</u>	<u>15.46</u>	1.71	13.26	10.19	1.67	<u>28.34</u>	43.99
DP-Generation (Kurakin et al., 2023)	16.22	10.94	<u>1.06</u>	14.97	11.20	1.05	25.51	42.75
DP-Instruct (Yu et al., 2024)	11.94	8.44	1.04	14.06	10.76	<u>1.04</u>	26.27	48.06
KnowledgeSG (Wang et al., 2024)	20.28	10.74	1.31	<u>24.14</u>	12.33	1.21	23.93	<u>49.58</u>
RewardDS	27.78	17.02	1.17	24.42	14.96	1.02	32.41	49.99
w/o Reward Guided Filtering	20.38	13.11	1.28	17.93	<u>12.52</u>	1.25	23.03	34.96
w/o Self-Optimizing Refinement	22.70	13.42	1.36	14.14	11.07	1.18	22.27	33.17

Table 1: Comparisons of our method with baselines across three domain-specific tasks: Medical QA, Financial QA, and Code Generation. Higher values of ROUGE-1 (R1) and ROUGE-L (RL), and lower values of Perplexity (PPL) indicate better performance on the QA generation task. Higher values of Pass@1 and Pass@10 reflect better performance in the code generation task. Numbers in **bold** and underlined represent the best and second-best results, respectively.

6.2 Compared Methods.

To demonstrate the effectiveness of our method, we consider several baselines for comparison:

Vanilla LLM refers to using a general-purpose LLM for domain-specific tasks without any domain adaptation or fine-tuning. **Locally Fine-tuning** refers to training a lightweight model locally on clients’ private data.

DP-Generation (Kurakin et al., 2023) fine-tunes the generation proxy model on the client side using DP-SGD. This proxy model is then used to generate synthetic data, which are subsequently utilized to fine-tune target LLM on the server. **DP-Instruct** (Yu et al., 2024) additionally filters synthetic data based on text similarity before LLM fine-tuning; **KnowledgeSG** (Wang et al., 2024) distills the capacity from LLM into the generation proxy model to enhance its performance.

More details of the compared method are provided in Appendix C.

6.3 Main Results

As shown in Table 1, *RewardDS* outperforms all other baselines across the three domain-specific tasks, except for the PPL on the Medical QA task. DP-Instruct achieves marginally lower PPL in medical QA. This may be attributed to the filtering strategy based on similarity, which could lead the target LLM to overfit on these highly similar samples.

The Vanilla LLM exhibits suboptimal performance across medical QA, financial QA, and code generation tasks, primarily due to the lack of domain-specific fine-tuning on private data. While Locally Fine-tuning a lightweight proxy model

(with only 0.5B parameters) mitigates privacy concerns, the small model’s limited capacity hinders its ability to effectively learn domain-specific knowledge, leading to subpar performance.

DP-Generation samples synthetic training data to fine-tune the target LLM on the server. However, due to the randomness inherent in the sampling process, the resulting synthetic data contains significant noise, which severely impairs the fine-tuning performance of the LLM on the server. DP-Instruct attempts to filter the synthetic data by computing the similarity between the synthetic query and the private query. But, similarity alone cannot accurately reflect the quality of synthetic data, where higher similarity does not necessarily indicate better data quality. KnowledgeSG distills the capabilities of the target LLM on the server into the generation proxy model for domain-specific tasks. However, since the target LLM is not specifically fine-tuned for these tasks, the improvement through distillation is limited.

We observe consistent performance declines across all tasks when either the Reward Guided Filtering or Self-Optimizing Refinement module is removed, highlighting the importance of both components. Without these components, more noisy synthetic samples are used during LLM fine-tuning, leading to degraded performance.

In addition, following Zheng et al. (2023), we employ an LLM-Judger to assess the generated response from our method and those baseline approaches for QA tasks. Specifically, we provide the LLM-Judger with responses from our method and those from baseline methods, prompting it to judge